

ECL: A Deterministic Execution IR for Replayable Agent Runtime Traces

Bin Zhang
Independent Researcher
Email: joy7759@gmail.com

Abstract—Agent runtimes emit execution traces through framework-specific structures such as model inputs, tool calls, callback trees, spans, and event lists. These records are useful inside a source framework, but they are difficult to compare, replay, or cite across runtimes. ECL is a small deterministic execution intermediate representation (IR) for replayable agent-runtime traces. The tool maps supported OpenAI-style and LangChain-style local fixtures into four ECL surfaces: state, intent, action, and evidence. It then validates the normalized record, emits replay artifacts, and computes stable hashes. This tool demonstration presents the repository, one-command reviewer workflow, Docker packaging path, replay outputs, and explicit loss reporting. ECL v0.1 is intentionally narrow: it is not an agent framework, tracing backend, benchmark, public standard, production deployment, or external adoption claim.

Index Terms—agent systems, execution traces, intermediate representation, deterministic replay, reproducibility

I. PROBLEM AND MOTIVATION

Agent frameworks increasingly expose traces for tool calls, model inputs, reasoning events, callbacks, and run trees. These traces help developers inspect individual runs, but their native structures remain tied to the originating runtime. A trace from an OpenAI-style agent and a trace from a LangChain-style run can both describe tool-using execution while still using different object models and event conventions.

ECL addresses a narrow software-engineering problem: provide a deterministic, local, replayable representation target for supported agent-runtime traces. The goal is not to replace existing tracing systems. The goal is to make a small execution record that can be validated, replayed, hashed, archived, and cited across runtime boundaries.

II. TOOL OVERVIEW

ECL v0.1 defines four execution surfaces: `state`, `intent`, `action`, and `evidence`. The repository includes a JSON schema, validators, deterministic JSON hashing utilities, OpenAI-style and LangChain-style adapters, an embeddable SDK, a dependency API, and a local MCP-shaped wrapper. The local wrapper is a protocol-surface simulation; it is not a published MCP server.

Table I summarizes the minimal mapping used in the demonstration.

Loss-aware mapping is part of the tool boundary. When source runtime information cannot be preserved completely, the adapter records loss information instead of claiming full-fidelity conversion.

TABLE I
ECL TRACE NORMALIZATION SURFACES

Runtime trace element	ECL surface
model input or run input	state
reasoning or prompt intent	intent
tool call or operation	action
events, results, loss report	evidence

III. REVIEWER WORKFLOW

The reviewer-facing local workflow is a single command:

```
make demo
```

This runs:

```
python3 -m unittest discover -s tests
python3 sdk/demo_dependency_mode.py
python3 examples/external_recognition_demo.py
```

The optional container path is:

```
docker build -t ecl-demo .
docker run --rm ecl-demo
```

The dependency-mode API used by the demo is:

```
import ecl_dependency as ecl

ecl_object = ecl.wrap(trace)
result = ecl.verify(ecl_object)
```

The demo uses bundled local fixtures and performs no external API calls.

IV. DEMONSTRATION SCENARIO

The video demonstration follows four steps. First, it opens the OpenAI-style and LangChain-style fixture files in `tests/fixtures/`. Second, it runs `make demo` and shows the passing tests and stable result hashes. Third, it opens the generated ECL objects for both runtimes and points to the four normalized surfaces. Fourth, it opens replay outputs, including the execution trace, evidence bundle, replay result, and verification hash.

The current checked local output is:

```
Ran 78 tests
OK
dependency_mode_result_hash=
sha256:b9d8fa0269bd2efef4572daed...
external_recognition_result_hash=
sha256:dfafe2572fdf1ee2f48732d0c...
```

Full hashes and generated artifacts are stored in the repository manifest files.

V. VALIDATION AND REPRODUCIBILITY

The verification surface includes unit tests for schema handling, adapters, SDK behavior, dependency-mode wrapping, local replay, and the MCP-shaped local wrapper. The repository also contains synthetic trace-corpus evaluation, negative validation-matrix evaluation, and mapping coverage reports. These are local reproducibility checks, not independent third-party validation.

The current Docker reviewer demo records an image identifier, a terminal transcript, and the same deterministic demo hashes as the host run. This supports the ICSE tool-demo requirement that reviewers should not have to perform complex manual setup.

VI. DISTRIBUTION

The software repository is publicly available on GitHub, and ECL v0.1 is archived as a Zenodo software record with DOI 10.5281/zenodo.21003766. The archive establishes a citable software object. It does not imply peer-review acceptance, community adoption, production deployment, or standardization.

VII. LIMITATIONS

ECL v0.1 supports only the included OpenAI-style and LangChain-style trace families. The current evaluation uses synthetic local fixtures. The replay model is deterministic for the local artifact surface, not a claim about deterministic behavior of external agent runtimes. The MCP-shaped wrapper is local only. The project does not claim benchmark superiority, production readiness, external adoption, or standards-body endorsement. A Journal of Systems and Software manuscript was rejected at pre-screening; the current ICSE tool-demo route is intended to obtain focused community feedback.

VIII. CONCLUSION

ECL demonstrates a minimal deterministic execution IR for replayable agent-runtime traces. The contribution is a small, runnable, DOI-archived tool artifact that normalizes supported traces into four surfaces, records mapping loss, emits replay artifacts, and verifies stable hashes through a one-command local workflow.

REFERENCES

- [1] B. Zhang, “ECL: Execution Compact Layer,” GitHub repository. [Online]. Available: <https://github.com/joy7758/ecl-execution-compact-layer>
- [2] B. Zhang, “ECL v0.1,” Zenodo software archive, 2026. doi: <https://doi.org/10.5281/zenodo.21003766>
- [3] ICSE 2027, “Tool Demonstration and Data Showcase.” [Online]. Available: <https://conf.researchr.org/track/icse-2027/icse-2027-demonstrations>
- [4] OpenTelemetry Authors, “OpenTelemetry documentation.” [Online]. Available: <https://opentelemetry.io/docs/>
- [5] W3C, “PROV Overview.” [Online]. Available: <https://www.w3.org/TR/prov-overview/>